

SR UNIVERSITY

AI Assisted Coding - Lab Assignment 13

Hospital Management System (BMI, Blood Pressure, Sugar Test)

Field	Details
Student Name	BITLA SAIRAJ
Roll No. / Hall Ticket No.	2503a52162
Program	B. Tech (CS & AI)
Department / Semester	Computer Science and AI / II
Batch	06
Faculty	Dr. Rajnish Kumar, Ms. B. Swathi
Assignment Date	24-09-2026
Tools Used	Visual Studio Code, Python 3.12.3, AI coding assistant (Claude)

Task 1: Refactoring - Eliminating Repetitive Logic (BMI)

2. AI Prompts Used

Prompt 1 - locate and refactor

*Act as a senior Python developer reviewing legacy code from a Hospital Management System.
Context: the script below calculates a patient's BMI for three patients by copy-pasting the same lines.*

Tasks:

- 1. Locate every place where the BMI computation is repeated and name the code smell.*
- 2. Refactor the repeated logic into one reusable function with a proper docstring (Args, Returns, Raises).*
- 3. Keep the printed output exactly the same: "Patient BMI: <value rounded to 2 decimals>".*
- 4. Do not add features I did not ask for. Explain every change you make.*

[paste legacy code]

Prompt 2 - verification

Write a small Python script that runs my original script and the refactored script and confirms that their printed output is identical.

4. Original (Legacy) Code and Output

Original code (task1_before.py)

```
# Legacy code: the same BMI calculation is copy-pasted for every patient
weight = 70
height = 1.75
bmi = weight / (height ** 2)
print("Patient BMI:", round(bmi, 2))

weight = 60
height = 1.60
bmi = weight / (height ** 2)
print("Patient BMI:", round(bmi, 2))

weight = 85
height = 1.70
bmi = weight / (height ** 2)
print("Patient BMI:", round(bmi, 2))
```

Output of the legacy script

```
Patient BMI: 22.86
Patient BMI: 23.44
Patient BMI: 29.41
```

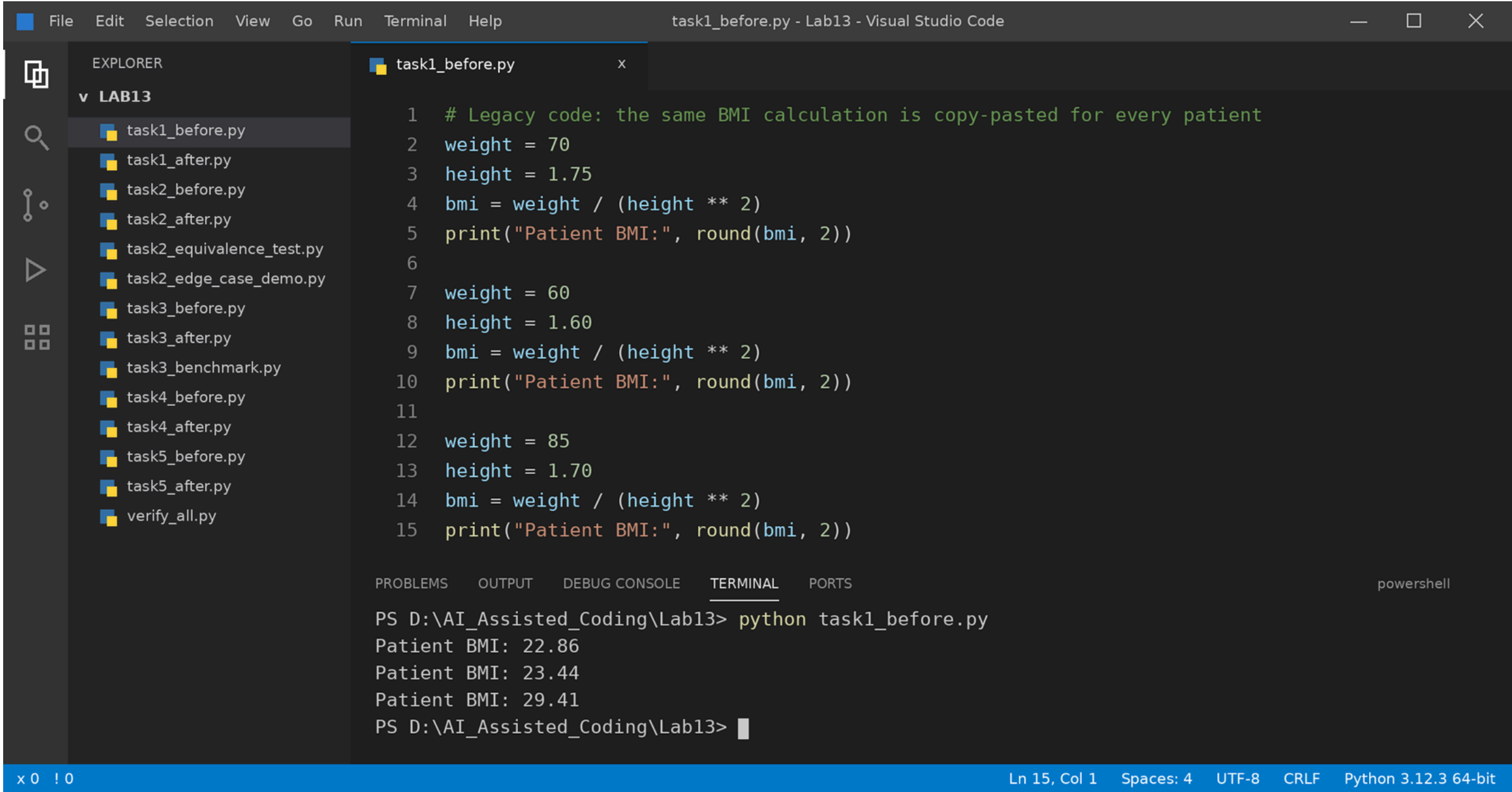


Figure 2: Legacy code (task1_before.py) and its output in VS Code

5. Refactored Code and Output

Refactored code (task1_after.py)

```
"""Hospital Management System - BMI module (refactored)."""

def calculate_bmi(weight_kg, height_m):
    """Calculate a patient's Body Mass Index (BMI).

    Args:
        weight_kg (float): Patient weight in kilograms.
        height_m (float): Patient height in metres (must be > 0).

    Returns:
        float: BMI value (weight / height squared), not rounded.

    Raises:
        ValueError: If height_m is zero or negative.
    """
    if height_m <= 0:
        raise ValueError("Height must be greater than zero.")
    return weight_kg / (height_m ** 2)

def print_patient_bmi(weight_kg, height_m):
    """Print a patient's BMI rounded to two decimal places.

    Args:
        weight_kg (float): Patient weight in kilograms.
        height_m (float): Patient height in metres.
    """
    print("Patient BMI:", round(calculate_bmi(weight_kg, height_m), 2))
```

```
def main():
    """Display the BMI of every patient in the sampled data."""
    patients = [
        (70, 1.75), # (weight in kg, height in m)
        (60, 1.60),
        (85, 1.70),
    ]
    for weight, height in patients:
        print_patient_bmi(weight, height)

if __name__ == "__main__":
    main()
```

Output of the refactored script

```
Patient BMI: 22.86
Patient BMI: 23.44
Patient BMI: 29.41
```

The screenshot shows the Visual Studio Code interface with the file explorer on the left displaying a directory named 'LAB13' containing various Python files. The main editor window shows the file 'task1_after.py' with the following code:

```

1  """Hospital Management System - BMI module (refactored)."""
2
3
4  def calculate_bmi(weight_kg, height_m):
5      """Calculate a patient's Body Mass Index (BMI).
6
7      Args:
8          weight_kg (float): Patient weight in kilograms.
9          height_m (float): Patient height in metres (must be > 0).
10
11      Returns:
12          float: BMI value (weight / height squared), not rounded.
13
14      Raises:
15          ValueError: If height_m is zero or negative.
16      """
17      if height_m <= 0:
18          raise ValueError("Height must be greater than zero.")
19      return weight_kg / (height_m ** 2)
20
21
22 def print_patient_bmi(weight_kg, height_m):
23     """Print a patient's BMI rounded to two decimal places.
24
25     Args:
26         weight_kg (float): Patient weight in kilograms.
27         height_m (float): Patient height in metres.
28     """
29     print("Patient BMI:", round(calculate_bmi(weight_kg, height_m), 2))
30
31
32 def main():
33     """Display the BMI of every patient in the sample data."""
34     patients = [
35         (70, 1.75), # (weight in kg, height in m)
36         (60, 1.60),
37         (85, 1.70),
38     ]
39     for weight, height in patients:
40         print_patient_bmi(weight, height)
41
42
43 if __name__ == "__main__":
44     main()

```

The terminal at the bottom shows the command 'python task1_after.py' being executed, resulting in the following output:

```

PS D:\AI_Assisted_Coding\Lab13> python task1_after.py
Patient BMI: 22.86
Patient BMI: 23.44
Patient BMI: 29.41
PS D:\AI_Assisted_Coding\Lab13>

```

Figure 3: Refactored code (task1_after.py) and its output in VS Code

6. Code Explanation -

- **calculate_bmi(weight_kg, height_m)** holds the single copy of the BMI formula. It returns the unrounded value, so it can be reused later (for example for the obesity check in Task 4).
- **print_patient_bmi(...)** keeps the presentation concern (rounding to 2 decimals and the "Patient BMI:" label) separate from the calculation.
- Patient data is now a list of (weight, height) tuples and a for loop prints one result per patient. Adding a new patient means adding one tuple.
- Every function has a docstring describing its arguments, return value and exceptions. main() plus the if `__name__ == "__main__"` guard lets the module be imported without running.
- A ValueError guard rejects a zero or negative height, which would otherwise cause a ZeroDivisionError or a meaningless BMI.

7. Verification (Output Unchanged)

Both scripts were executed and produced identical output: 22.86, 23.44 and 29.41 (the same three lines). The automated comparison (verify_all.py, shown in the Verification Summary at the end) reports PASS for Task 1.

Task 2: Refactoring - Modularizing Inline Calculations (Blood Pressure)

2. AI Prompts Used

Prompt 1 - extract the function

Act as a senior Python developer. The script below classifies a patient's blood pressure with an inline if/elif/else chain that is copy-pasted for each patient.

1. Identify the repeated blood-pressure classification pattern.
2. Convert it into a single reusable function that RETURNS the classification string instead of printing it.
3. Add a docstring and keep the code readable.
4. The output for every patient must remain identical. Do not change any threshold or operator.

[paste legacy code]

Prompt 2 - equivalence and review

Write a test that compares the legacy if/elif logic with the new function over a wide range of systolic and diastolic values. Then review the rules for logical problems WITHOUT changing my refactor, and tell me which readings would be categorised differently from standard blood-pressure categories.

4. Original (Legacy) Code and Output

Original code (task2_before.py)

```
# Legacy code: the same blood-pressure if/elif block is repeated per patient
systolic=130
diastolic=85
if systolic<120 and diastolic<80:
    print("Normal")
elif systolic<130 and diastolic<80:
    print("Elevated")
elif systolic<140 or diastolic<90:
    print("High Blood Pressure - Stage1")
else:
    print("High Blood Pressure - Stage2")

systolic=145
diastolic=95
if systolic<120 and diastolic<80:
    print("Normal")
elif systolic<130 and diastolic<80:
    print("Elevated")
elif systolic<140 or diastolic<90:
    print("High Blood Pressure - Stage1")
else:
    print("High Blood Pressure - Stage2")
```

Output of the legacy script

HighBlood Pressure- Stage1
HighBlood Pressure- Stage2

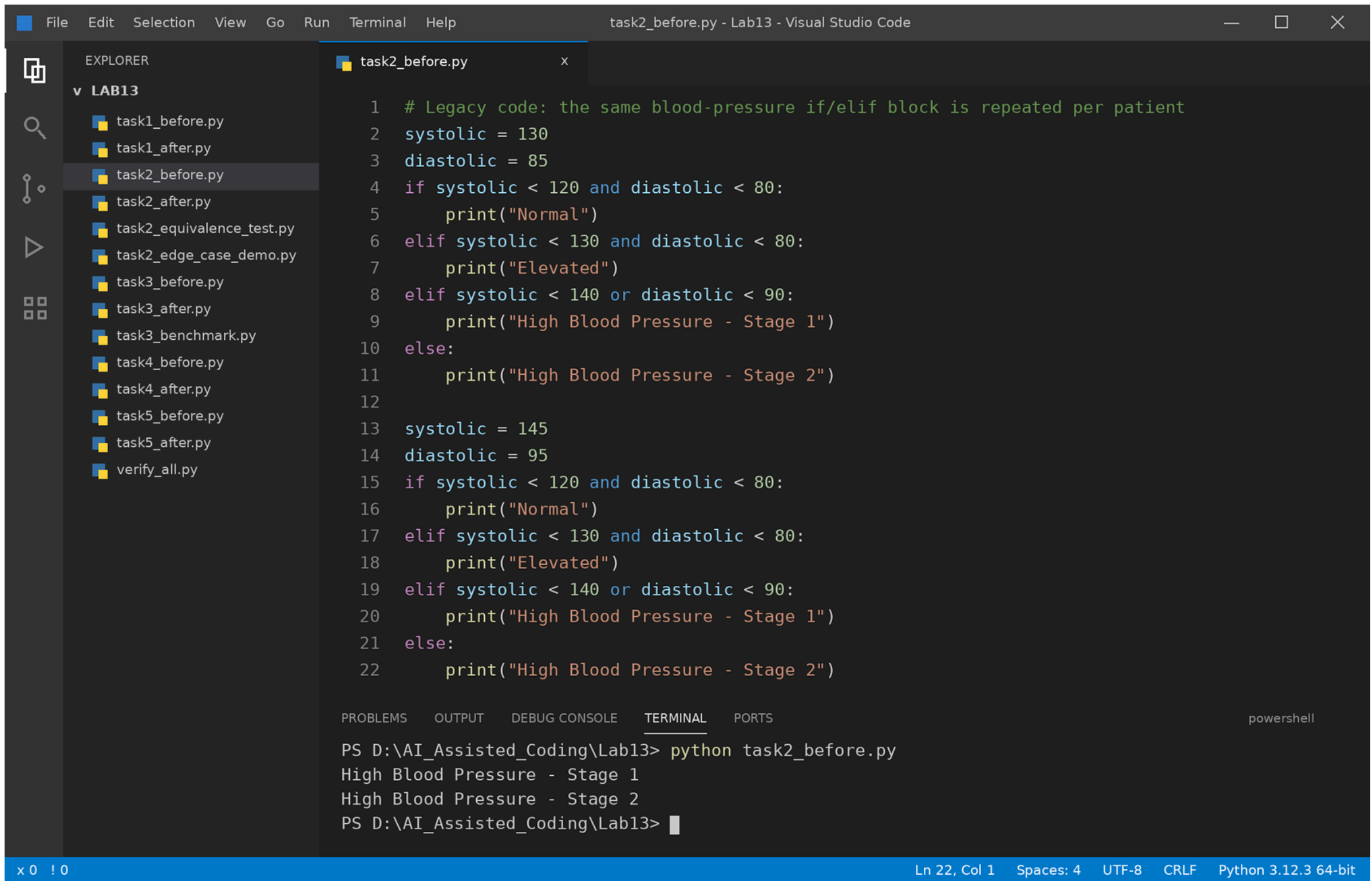


Figure 4: Legacy code (task2_before.py) and its output in VS Code

5. Refactored Code and Output

Refactored code (task2_after.py)

```
"""Hospital Management System - blood pressure module (refactored)."""

def classify_blood_pressure(systolic, diastolic):
    """Classify a blood pressure reading.

    The rules are evaluated in order and the first match wins
    (identical to the original inline if/elif chain).

    Args:
        systolic (int): Systolic pressure in mmHg (upper number).
        diastolic (int): Diastolic pressure in mmHg (lower number).

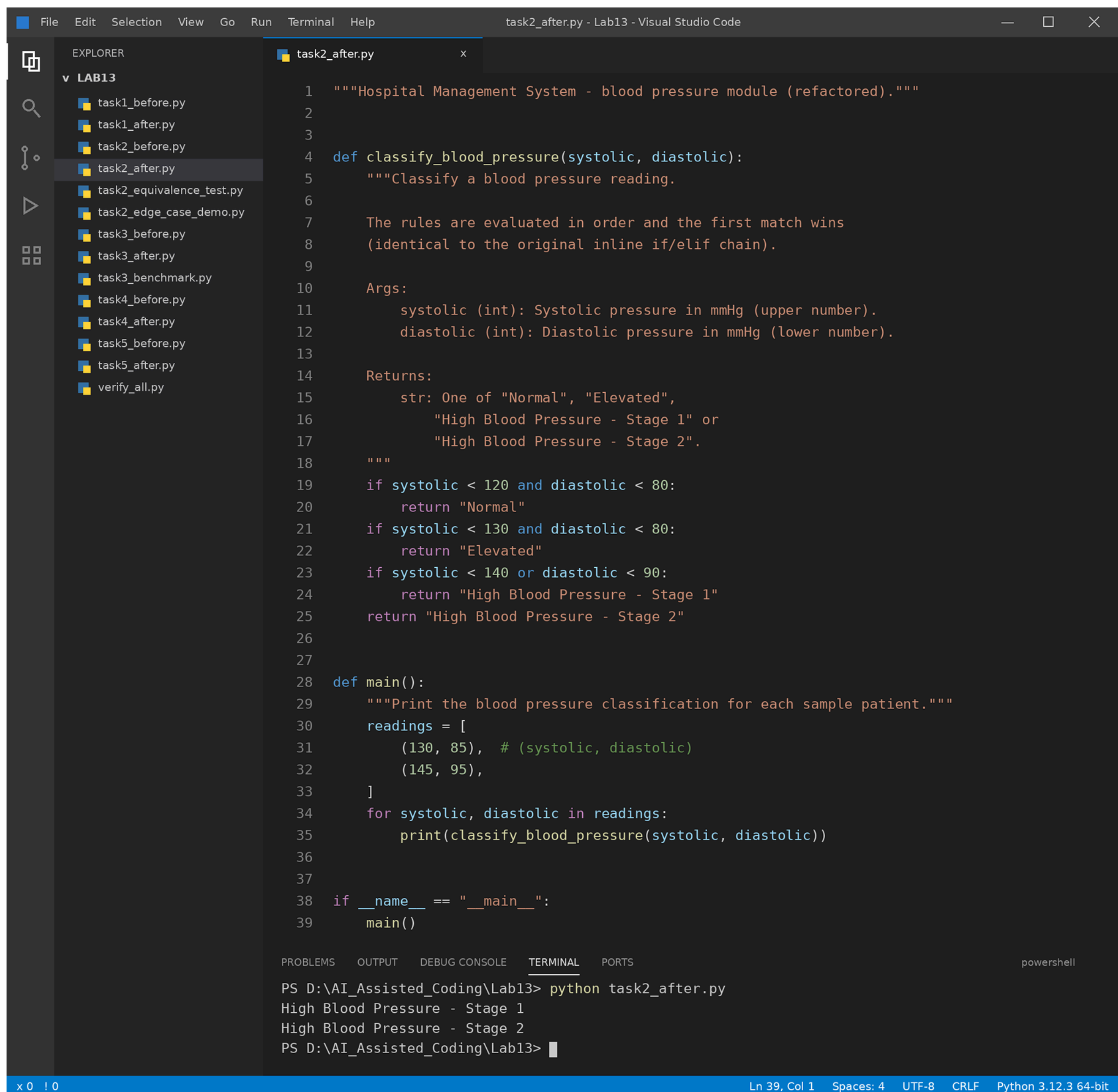
    Returns:
        str: One of "Normal", "Elevated",
            "High Blood Pressure - Stage 1" or
            "High Blood Pressure - Stage 2".
    """
    if systolic < 120 and diastolic < 80:
        return "Normal"
    if systolic < 130 and diastolic < 80:
        return "Elevated"
    if systolic < 140 or diastolic < 90:
        return "High Blood Pressure - Stage 1"
    return "High Blood Pressure - Stage 2"
```

```
def main():
    """Print the blood pressure classification for each sample patient."""
    readings = [
        (130, 85), #(systolic, diastolic)
        (145, 95),
    ]
    for systolic, diastolic in readings:
        print(classify_blood_pressure(systolic, diastolic))

if __name__ == "__main__":
    main()
```

Output of the refactored script

```
HighBlood Pressure- Stage 1
HighBlood Pressure- Stage 2
```



The screenshot displays the Visual Studio Code interface with the file `task2_after.py` open in the editor. The code is a Python script for a blood pressure classification module. It includes a `classify_blood_pressure` function that takes systolic and diastolic pressure values and returns a classification string based on specific thresholds. The `main` function uses this classification function to process a list of sample readings. The terminal at the bottom shows the command `python task2_after.py` being executed, resulting in the output: `High Blood Pressure - Stage 1` and `High Blood Pressure - Stage 2`.

```
1  """Hospital Management System - blood pressure module (refactored)."""
2
3
4  def classify_blood_pressure(systolic, diastolic):
5      """Classify a blood pressure reading.
6
7      The rules are evaluated in order and the first match wins
8      (identical to the original inline if/elif chain).
9
10     Args:
11         systolic (int): Systolic pressure in mmHg (upper number).
12         diastolic (int): Diastolic pressure in mmHg (lower number).
13
14     Returns:
15         str: One of "Normal", "Elevated",
16             "High Blood Pressure - Stage 1" or
17             "High Blood Pressure - Stage 2".
18     """
19     if systolic < 120 and diastolic < 80:
20         return "Normal"
21     if systolic < 130 and diastolic < 80:
22         return "Elevated"
23     if systolic < 140 or diastolic < 90:
24         return "High Blood Pressure - Stage 1"
25     return "High Blood Pressure - Stage 2"
26
27
28 def main():
29     """Print the blood pressure classification for each sample patient."""
30     readings = [
31         (130, 85), #(systolic, diastolic)
32         (145, 95),
33     ]
34     for systolic, diastolic in readings:
35         print(classify_blood_pressure(systolic, diastolic))
36
37
38 if __name__ == "__main__":
39     main()
```

```
PS D:\AI_Assisted_Coding\Lab13> python task2_after.py
High Blood Pressure - Stage 1
High Blood Pressure - Stage 2
PS D:\AI_Assisted_Coding\Lab13>
```

Figure 5: Refactored code (`task2_after.py`) and its output in VS Code

5a. Equivalence Test

To go beyond comparing two sample patients, the refactored function was compared with a copy of the legacy chain for every combination of systolic 80-200 mmHg and diastolic 40-130 mmHg (11,011 readings).

task2_equivalence_test.py

```
"""Prove the refactored function behaves exactly like the legacy inline code."""
from task2_after import classify_blood_pressure

def legacy_inline_classification(systolic, diastolic):
    """Original if/elif chain, kept only for comparison."""
    if systolic < 120 and diastolic < 80:
        result = "Normal"
    elif systolic < 130 and diastolic < 80:
        result = "Elevated"
    elif systolic < 140 or diastolic < 90:
        result = "High Blood Pressure - Stage 1"
    else:
        result = "High Blood Pressure - Stage 2"
    return result

checked = 0
for systolic in range(80, 201):
    for diastolic in range(40, 131):
        assert classify_blood_pressure(systolic, diastolic) == \
            legacy_inline_classification(systolic, diastolic)
        checked += 1

print(f"Compared {checked} systolic/diastolic combinations.")
print("Result: PASS - refactored function matches the legacy logic.")
```

Output

```
Compared 11011 systolic/diastolic combinations.
Result: PASS - refactored function matches the legacy logic.
```

```

1  """Prove the refactored function behaves exactly like the legacy inline code."""
2  from task2_after import classify_blood_pressure
3
4
5  def legacy_inline_classification(systolic, diastolic):
6      """Original if/elif chain, kept only for comparison."""
7      if systolic < 120 and diastolic < 80:
8          result = "Normal"
9      elif systolic < 130 and diastolic < 80:
10         result = "Elevated"
11     elif systolic < 140 or diastolic < 90:
12         result = "High Blood Pressure - Stage 1"
13     else:
14         result = "High Blood Pressure - Stage 2"
15     return result
16
17
18     checked = 0
19     for systolic in range(80, 201):
20         for diastolic in range(40, 131):
21             assert classify_blood_pressure(systolic, diastolic) == \
22                 legacy_inline_classification(systolic, diastolic)
23             checked += 1
24
25     print(f"Compared {checked} systolic/diastolic combinations.")
26     print("Result: PASS - refactored function matches the legacy logic.")

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

PS D:\AI_Assisted_Coding\Lab13> python task2_equivalence_test.py
Compared 11011 systolic/diastolic combinations.
Result: PASS - refactored function matches the legacy logic.
PS D:\AI_Assisted_Coding\Lab13>

Figure 6: Equivalence test: refactored function vs legacy logic

5b. Observation About the Legacy Rule

Fact: In the legacy chain the third condition is `systolic < 140 or diastolic < 90`. Because of the `or`, a reading such as 160/70 satisfies `diastolic < 90` and is printed as "High Blood Pressure - Stage 1".

Fact: The 2017 ACC/AHA blood-pressure categories place a systolic value of 140 or more in Stage 2, and they use the higher category when the two numbers disagree (general knowledge, not looked up for this report).

Inference: the `or` looks like a simplification in the lab sample rather than a deliberate clinical rule, but the assignment does not say so. Because the lab requires unchanged output, the refactored function keeps the legacy behaviour exactly. The demo below only illustrates the difference and is educational, not medical advice.

task2_edge_case_demo.py

```

"""Review note: legacy rule vs ACC/AHA 2017 categories (educational only)."""
from task2_after import classify_blood_pressure

def guideline_category(systolic, diastolic):
    """Stage rules where EITHER number can place a patient in a higher stage."""
    if systolic >= 140 or diastolic >= 90:
        return "High Blood Pressure - Stage 2"
    if systolic >= 130 or diastolic >= 80:
        return "High Blood Pressure - Stage 1"
    if systolic >= 120:
        return "Elevated"

```

```
return "Normal"
```

```
for sys_bp, dia_bp in [(160, 70), (150, 85), (130, 85), (145, 95)]:  
    legacy = classify_blood_pressure(sys_bp, dia_bp)  
    guide = guideline_category(sys_bp, dia_bp)  
    flag = "same" if legacy == guide else "DIFFERENT"  
    print(f"{sys_bp}/{dia_bp} legacy: {legacy}")  
    print(f"{'':8}guideline: {guide} [{flag}]")
```

Output

```
160/70 legacy: High Blood Pressure - Stage 1  
      guideline: High Blood Pressure - Stage 2 [DIFFERENT]  
150/85 legacy: High Blood Pressure - Stage 1  
      guideline: High Blood Pressure - Stage 2 [DIFFERENT]  
130/85 legacy: High Blood Pressure - Stage 1  
      guideline: High Blood Pressure - Stage 1 [same]  
145/95 legacy: High Blood Pressure - Stage 2  
      guideline: High Blood Pressure - Stage 2 [same]
```

The screenshot shows the Visual Studio Code editor with the file `task2_edge_case_demo.py` open. The file contains a Python script that compares legacy blood pressure classification rules with guideline-style rules for four specific edge-case readings. The script defines a `guideline_category` function and a loop that iterates over the readings, calling `classify_blood_pressure` and `guideline_category` to determine the legacy and guideline categories, and then prints the results with a flag indicating if they are the same or different.

```
1 """Review note: legacy rule vs ACC/AHA 2017 categories (educational only)."""  
2 from task2_after import classify_blood_pressure  
3  
4  
5 def guideline_category(systolic, diastolic):  
6     """Stage rules where EITHER number can place a patient in a higher stage."""  
7     if systolic >= 140 or diastolic >= 90:  
8         return "High Blood Pressure - Stage 2"  
9     if systolic >= 130 or diastolic >= 80:  
10        return "High Blood Pressure - Stage 1"  
11    if systolic >= 120:  
12        return "Elevated"  
13    return "Normal"  
14  
15  
16 for sys_bp, dia_bp in [(160, 70), (150, 85), (130, 85), (145, 95)]:  
17     legacy = classify_blood_pressure(sys_bp, dia_bp)  
18     guide = guideline_category(sys_bp, dia_bp)  
19     flag = "same" if legacy == guide else "DIFFERENT"  
20     print(f"{sys_bp}/{dia_bp} legacy: {legacy}")  
21     print(f"{'':8}guideline: {guide} [{flag}]")
```

The terminal output shows the results of running the script:

```
PS D:\AI_Assisted_Coding\Lab13> python task2_edge_case_demo.py  
160/70 legacy: High Blood Pressure - Stage 1  
      guideline: High Blood Pressure - Stage 2 [DIFFERENT]  
150/85 legacy: High Blood Pressure - Stage 1  
      guideline: High Blood Pressure - Stage 2 [DIFFERENT]  
130/85 legacy: High Blood Pressure - Stage 1  
      guideline: High Blood Pressure - Stage 1 [same]  
145/95 legacy: High Blood Pressure - Stage 2  
      guideline: High Blood Pressure - Stage 2 [same]  
PS D:\AI_Assisted_Coding\Lab13>
```

Figure 7: Legacy rule versus a guideline-style rule for edge-case readings

6. Code Explanation

- **`classify_blood_pressure(systolic, diastolic)`** contains the one and only copy of the rules and returns a string. The caller decides what to do with it (print, store, test).

- The conditions, their order and the returned labels are exactly those of the original chain. The elif keywords became early return statements, which are equivalent because each branch ends the function.
- Patient readings are stored as (systolic, diastolic) tuples and processed in a loop, so a new patient needs one extra tuple.
- The docstring documents the parameters, the return values and the first-match-wins evaluation order, which is easy to miss when reading the code.

7. Verification (Output Unchanged)

The two sample patients print the same output before and after: 130/85 gives "High Blood Pressure - Stage 1" and 145/95 gives "High Blood Pressure - Stage 2". The exhaustive comparison above found no difference across 11,011 readings, and `verify_all.py` reports PASS for Task 2.

Task 3: Refactoring - Improving Loop and Conditional Performance (Sugar Test)

2. AI Prompts Used

Prompt 1 - recommend and refactor

Act as a Python performance reviewer. The code below uses nested loops to check whether each patient scheduled for a sugar test is registered.

- 1. Explain the time complexity and the inefficiency.*
- 2. Recommend a more efficient data structure (for example a set) and refactor the code to use it.*
- 3. Preserve the program behaviour: same messages, same order.*
- 4. Write a benchmark with timeit that compares the old and new approach on the sample data and on a large dataset.*

Note: the pasted code lost its indentation; assume the if/else runs once per name after the inner loop finishes.

[paste legacy code]

4. Original (Legacy) Code and Output

Original code (task3_before.py)

```
# Legacy code: nested loops to check whether each patient is registered
patients = ["Ravi", "Sita", "Kiran", "Meena"]
sugar_test_list = ["Kiran", "Anil", "Sita"]

for name in sugar_test_list:
    exists = False
    for patient in patients:
        if name == patient:
            exists = True
    if exists:
        print(name, "- record found, proceed with sugar test")
    else:
        print(name, "- not registered, please register before test")
```

Output of the legacy script

```
Kiran - record found, proceed with sugar test
Anil - not registered, please register before test
Sita - record found, proceed with sugar test
```

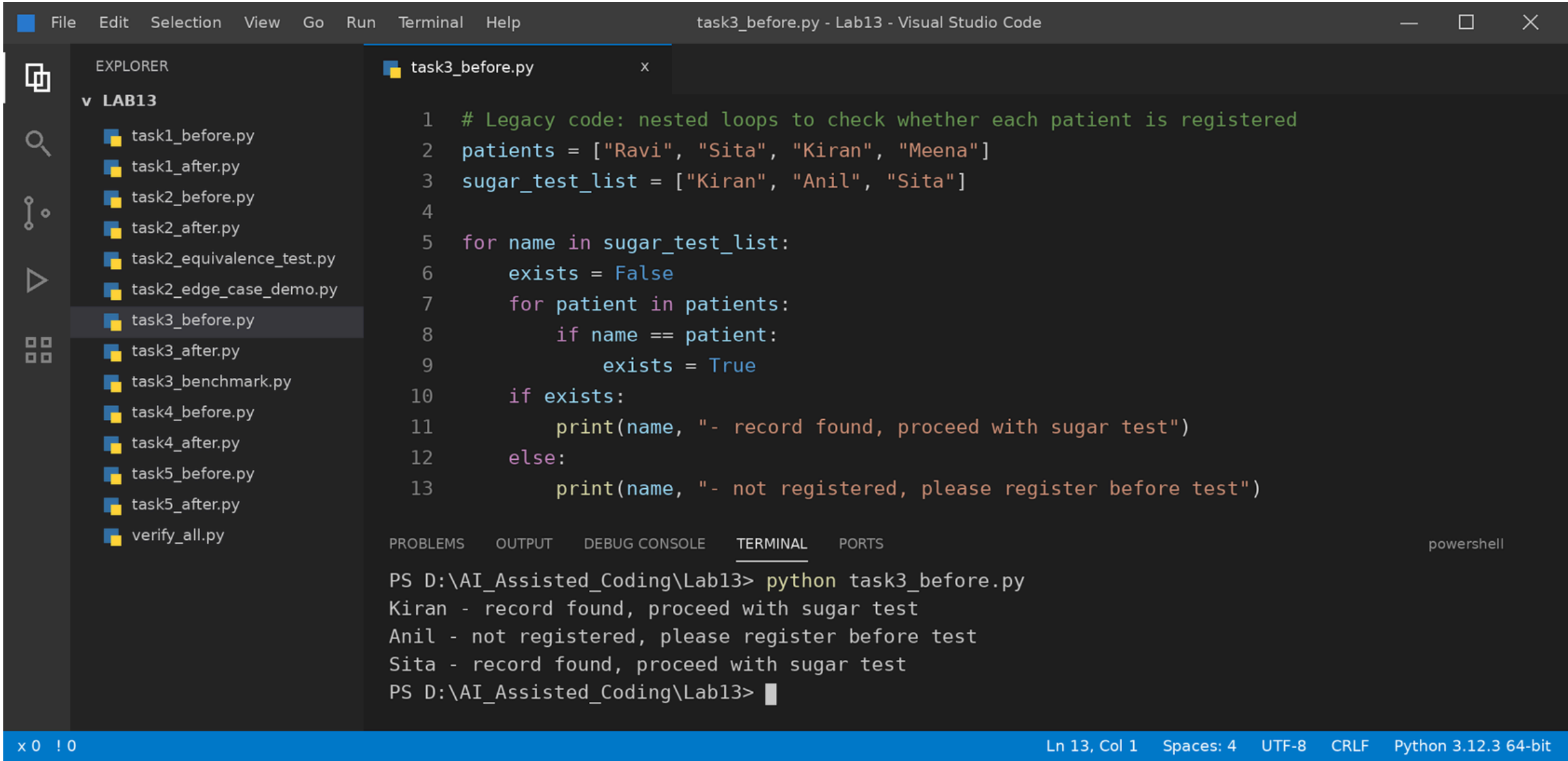


Figure 8: Legacy code (task3_before.py) and its output in VS Code

5. Refactored Code and Output

Refactored code (task3_after.py)

```
"""Hospital Management System - sugartest registration check (optimised)."""

def check_sugar_test_registrations(patients,sugar_test_list):
    """Report whether each patient scheduled for a sugar test is registered.

    A set is built once from the patient records so each lookup takes
    O(1) time on average, instead of scanning the whole list every time.

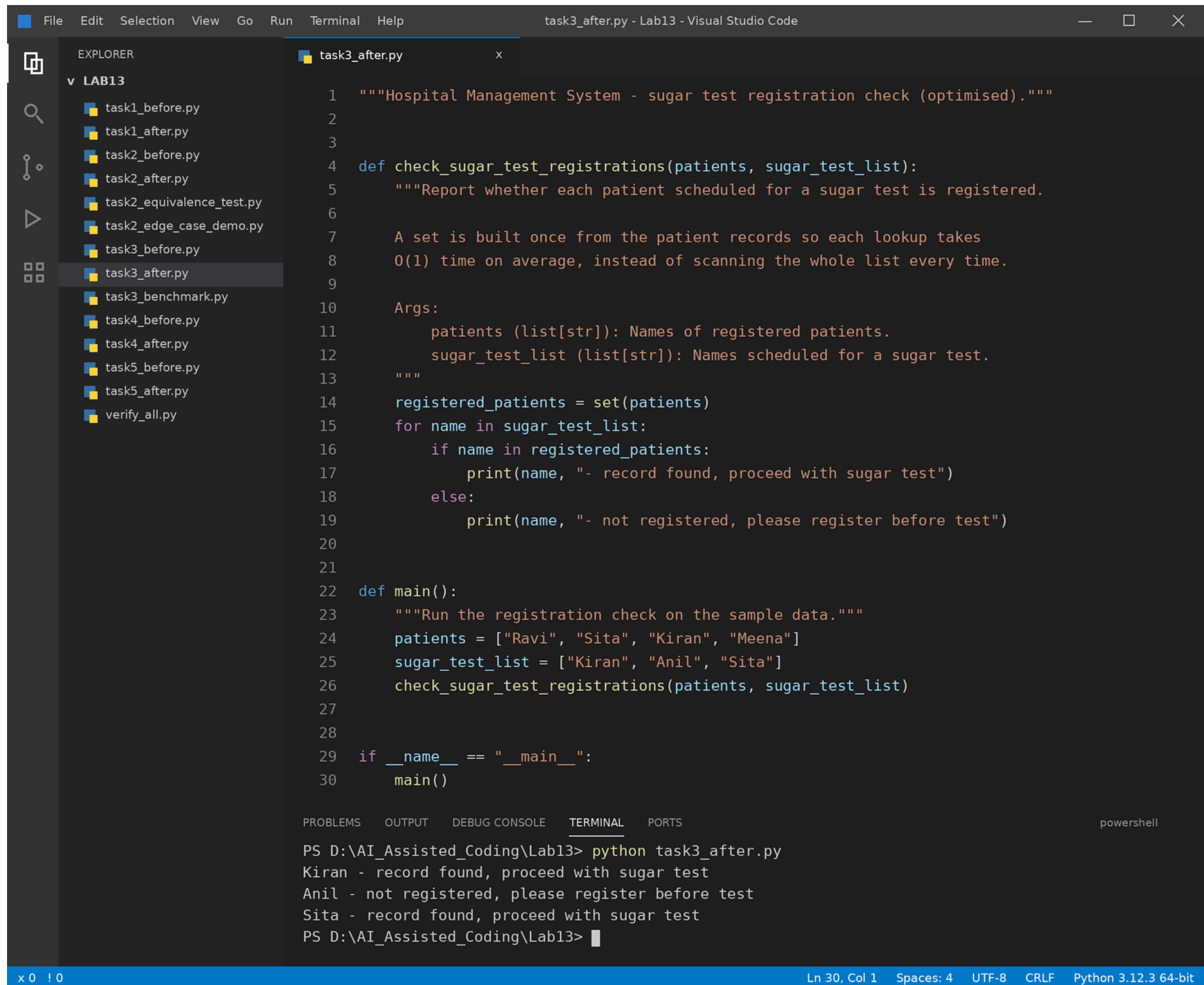
    Args:
        patients (list[str]): Names of registered patients.
        sugar_test_list (list[str]): Names scheduled for a sugartest.
    """
    registered_patients = set(patients)
    for name in sugar_test_list:
        if name in registered_patients:
            print(name, "- record found, proceed with sugar test")
        else:
            print(name, "- not registered, please register before test")

def main():
    """Run the registration check on the sample data."""
    patients = ["Ravi", "Sita", "Kiran", "Meena"]
    sugar_test_list = ["Kiran", "Anil", "Sita"]
    check_sugar_test_registrations(patients,sugar_test_list)

if __name__ == "__main__":
    main()
```

Output of the refactored script

Kiran - record found, proceed with sugar test
Anil - not registered, please register before test
Sita - record found, proceed with sugar test



The screenshot shows the Visual Studio Code interface with the file explorer on the left displaying a directory named 'LAB13' containing various task files. The main editor window shows the code for 'task3_after.py'. The code defines a function 'check_sugar_test_registrations' that uses a set to efficiently check if patients are registered. The 'main' function uses sample data to test this function. The terminal at the bottom shows the command 'python task3_after.py' and its output, which matches the expected results from the text above.

```
1 """Hospital Management System - sugar test registration check (optimised)."""
2
3
4 def check_sugar_test_registrations(patients, sugar_test_list):
5     """Report whether each patient scheduled for a sugar test is registered.
6
7     A set is built once from the patient records so each lookup takes
8     O(1) time on average, instead of scanning the whole list every time.
9
10    Args:
11        patients (list[str]): Names of registered patients.
12        sugar_test_list (list[str]): Names scheduled for a sugar test.
13    """
14    registered_patients = set(patients)
15    for name in sugar_test_list:
16        if name in registered_patients:
17            print(name, "- record found, proceed with sugar test")
18        else:
19            print(name, "- not registered, please register before test")
20
21
22 def main():
23     """Run the registration check on the sample data."""
24     patients = ["Ravi", "Sita", "Kiran", "Meena"]
25     sugar_test_list = ["Kiran", "Anil", "Sita"]
26     check_sugar_test_registrations(patients, sugar_test_list)
27
28
29 if __name__ == "__main__":
30     main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS D:\AI_Assisted_Coding\Lab13> python task3_after.py
Kiran - record found, proceed with sugar test
Anil - not registered, please register before test
Sita - record found, proceed with sugar test
PS D:\AI_Assisted_Coding\Lab13>
```

x 0 ! 0 Ln 30, Col 1 Spaces: 4 UTF-8 CRLF Python 3.12.3 64-bit

Figure 9: Refactored code (task3_after.py) and its output in VS Code

5a. Execution Efficiency Comparison

task3_benchmark.py times both approaches with timeit. It first asserts that both return identical results, then reports the average time per run. Two datasets are used: the original sample (4 patients, 3 tests) and a generated large dataset (2,000 patients, 1,500 scheduled tests).

task3_benchmark.py

```
"""Compare lookup strategies: nested loops vs list 'in' vs set."""
import timeit

def lookup_nested_loops(patients, tests):
    """Legacy approach: scan the whole patient list for every name."""
    results = []
    for name in tests:
        exists = False
        for patient in patients:
            if name == patient:
                exists = True
        results.append(exists)
```

```

return results

def lookup_set(patients, tests):
    """Refactored approach: build a set once, then O(1) membership tests."""
    registered = set(patients)
    return [name in registered for name in tests]

def average_microseconds(func, patients, tests, runs):
    """Return the average run time of func in microseconds."""
    seconds = timeit.timeit(lambda: func(patients, tests), number=runs)
    return seconds / runs * 1_000_000

def compare(title, patients, tests, nested_runs, set_runs):
    """Time both approaches on the same data and print a summary."""
    assert lookup_nested_loops(patients, tests) == lookup_set(patients, tests)
    nested = average_microseconds(lookup_nested_loops, patients, tests, nested_runs)
    fast = average_microseconds(lookup_set, patients, tests, set_runs)
    print(f"{title}: {len(patients)} patients, {len(tests)} tests")
    print(f"  Nested loops: {nested:12.2f} us per run")
    print(f"  Set lookup   : {fast:12.2f} us per run")
    print(f"  Speed-up    : {nested / fast:12.1f}x")

small_patients = ["Ravi", "Sita", "Kiran", "Meena"]
small_tests = ["Kiran", "Anil", "Sita"]
compare("Sample data", small_patients, small_tests, 200_000, 200_000)

big_patients = [f"Patient_{i}" for i in range(2000)]
big_tests = [f"Patient_{i*3}" for i in range(1500)]
compare("Large data", big_patients, big_tests, 3, 300)

```

Output

```

Sample data: 4 patients, 3 tests
  Nested loops:    0.34 us per run
  Set lookup   :    0.33 us per run
  Speed-up    :     1.0x
Large data: 2000 patients, 1500 tests
  Nested loops: 44182.40 us per run
  Set lookup   :   93.65 us per run
  Speed-up    :   471.8x

```

The screenshot shows a Visual Studio Code editor with a file explorer on the left and a code editor in the center. The file explorer shows a directory named 'LAB13' containing several Python files. The code editor displays the contents of 'task3_benchmark.py', which is a Python script comparing two lookup strategies: nested loops and set lookup. The script includes functions for benchmarking and a main execution block. The terminal at the bottom shows the output of running the script, which compares the performance of the two methods on both small and large datasets.

```

1  """Compare lookup strategies: nested loops vs list 'in' vs set."""
2  import timeit
3
4
5  def lookup_nested_loops(patients, tests):
6      """Legacy approach: scan the whole patient list for every name."""
7      results = []
8      for name in tests:
9          exists = False
10         for patient in patients:
11             if name == patient:
12                 exists = True
13             results.append(exists)
14     return results
15
16
17 def lookup_set(patients, tests):
18     """Refactored approach: build a set once, then O(1) membership tests."""
19     registered = set(patients)
20     return [name in registered for name in tests]
21
22
23 def average_microseconds(func, patients, tests, runs):
24     """Return the average run time of func in microseconds."""
25     seconds = timeit.timeit(lambda: func(patients, tests), number=runs)
26     return seconds / runs * 1_000_000
27
28
29 def compare(title, patients, tests, nested_runs, set_runs):
30     """Time both approaches on the same data and print a summary."""
31     assert lookup_nested_loops(patients, tests) == lookup_set(patients, tests)
32     nested = average_microseconds(lookup_nested_loops, patients, tests, nested_runs)
33     fast = average_microseconds(lookup_set, patients, tests, set_runs)
34     print(f"{title}: {len(patients)} patients, {len(tests)} tests")
35     print(f"  Nested loops : {nested:12.2f} us per run")
36     print(f"  Set lookup   : {fast:12.2f} us per run")
37     print(f"  Speed-up    : {nested / fast:12.1f} x")
38
39
40 small_patients = ["Ravi", "Sita", "Kiran", "Meena"]
41 small_tests = ["Kiran", "Anil", "Sita"]
42 compare("Sample data", small_patients, small_tests, 200_000, 200_000)
43
44 big_patients = [f"Patient_{i}" for i in range(2000)]
45 big_tests = [f"Patient_{i * 3}" for i in range(1500)]
46 compare("Large data ", big_patients, big_tests, 3, 300)

```

```

PS D:\AI_Assisted_Coding\Lab13> python task3_benchmark.py
Sample data: 4 patients, 3 tests
  Nested loops :      0.34 us per run
  Set lookup   :      0.33 us per run
  Speed-up    :       1.0 x
Large data : 2000 patients, 1500 tests
  Nested loops :  44182.40 us per run
  Set lookup   :    93.65 us per run
  Speed-up    :   471.8 x
PS D:\AI_Assisted_Coding\Lab13>

```

Figure 10: Benchmark: nested loops vs set lookup

6. Code Explanation

- **set(patients)** is built once; checking name in registered_patients is $O(1)$ on average, so the total cost drops from $O(n \times m)$ to $O(n + m)$.
- The inner loop and the exists flag are gone; the if/else now reads directly as the business rule.
- Messages and processing order follow sugar_test_list exactly as before, so the printed lines are identical.

- The logic lives in `check_sugar_test_registrations()`, documented with a docstring, and `main()` supplies the sample data.
- **Trade-offs:** building the set costs $O(n)$ time and extra memory; it pays off when there are many lookups or many patients. Set items must be hashable (strings are fine), and a set drops duplicate names and ordering, which does not matter for a membership test.

7. Verification (Output Unchanged)

Both versions print: "Kiran - record found...", "Anil - not registered...", "Sita - record found...". `verify_all.py` reports PASS for Task 3, and the benchmark asserts equal results on both datasets before timing.

Task 4: Refactoring - Replacing Hardcoded Values with Constants

2. AI Prompts Used

Prompt 1 - constants

Act as a senior Python developer. In the code below, detect every hardcoded numeric threshold used directly in a classification expression.

1. Define descriptive UPPER_CASE constants at the top of the program (for example HIGH_BP_SYSTOLIC, DIABETIC_SUGAR_LEVEL, OBESE_BMI_LIMIT), with a short comment giving the unit.
2. Update the expressions to use the constants so that the results do not change.
3. Add docstrings and keep the comparison operators exactly as they are.

[paste legacy code]

4. Original (Legacy) Code and Output

Original code (task4_before.py)

```
# Legacy code: threshold values are hardcoded inside the expressions
print("BP Status:", "High" if 145 > 140 else "Normal")
print("BP Status:", "High" if 138 > 140 else "Normal")
print("Sugar Status:", "Diabetic" if 180 > 126 else "Normal")
print("Sugar Status:", "Diabetic" if 110 > 126 else "Normal")
print("BMI Status:", "Obese" if 31.5 > 30 else "Normal")
```

Output of the legacy script

```
BP Status: High
BP Status: Normal
Sugar Status: Diabetic
Sugar Status: Normal
BMI Status: Obese
```

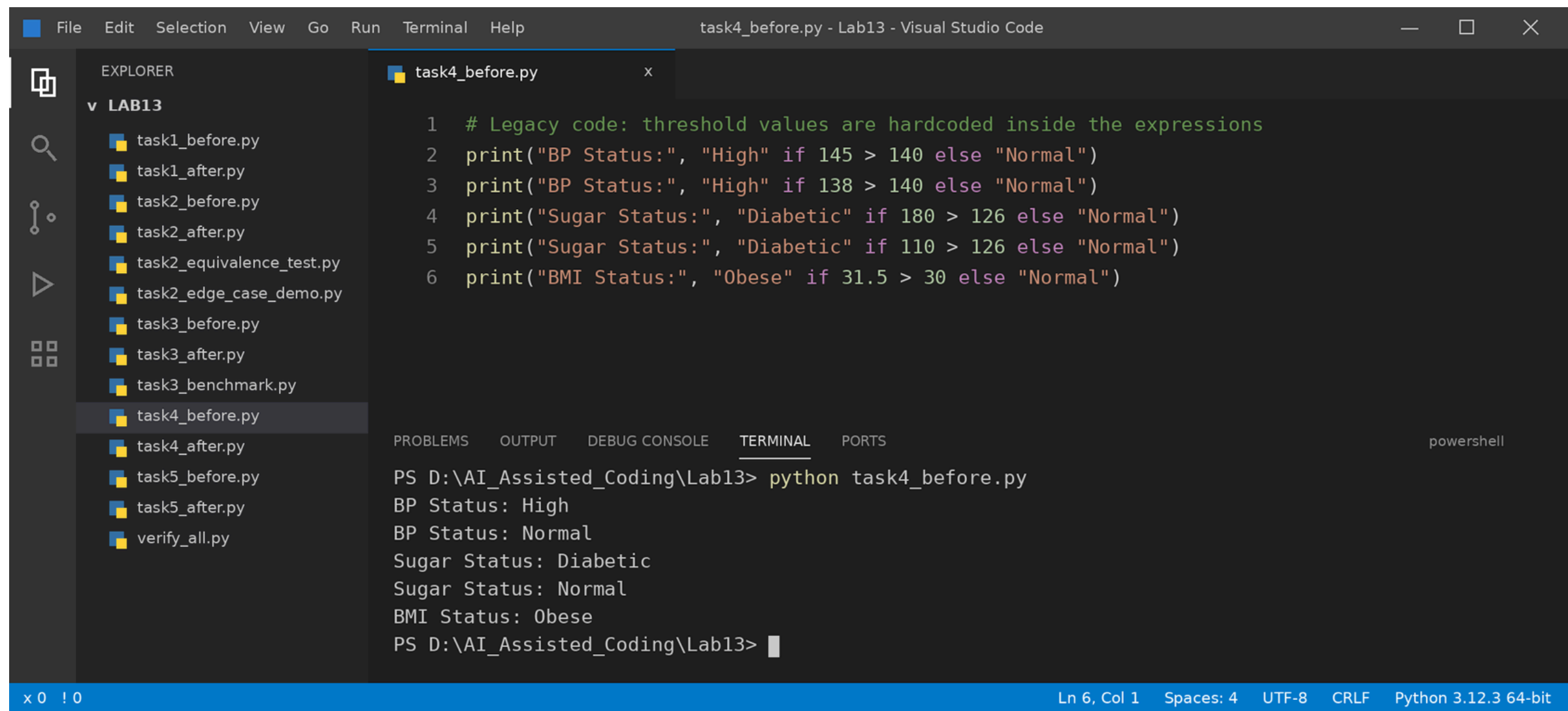


Figure 11: Legacy code (task4_before.py) and its output in VS Code

5. Refactored Code and Output

Refactored code (task4_after.py)

```
"""Hospital Management System - health status checks using named constants."""

# ---- Clinical threshold constants (change them here only) ----
HIGH_BP_SYSTOLIC=140    # systolic pressure above this is "High" (mmHg)
DIABETIC_SUGAR_LEVEL=126 # sugar level above this is "Diabetic" (mg/dL)
OBESE_BMI_LIMIT=30      # BMI above this is "Obese" (kg/m^2)

def get_bp_status(systolic):
    """Return "High" if systolic pressure exceeds HIGH_BP_SYSTOLIC, else "Normal"."""
    return "High" if systolic > HIGH_BP_SYSTOLIC else "Normal"

def get_sugar_status(sugar_level):
    """Return "Diabetic" if sugar exceeds DIABETIC_SUGAR_LEVEL, else "Normal"."""
    return "Diabetic" if sugar_level > DIABETIC_SUGAR_LEVEL else "Normal"

def get_bmi_status(bmi):
    """Return "Obese" if BMI exceeds OBESE_BMI_LIMIT, else "Normal"."""
    return "Obese" if bmi > OBESE_BMI_LIMIT else "Normal"

def main():
    """Print the status for each sample reading."""
    print("BP Status:", get_bp_status(145))
    print("BP Status:", get_bp_status(138))
    print("Sugar Status:", get_sugar_status(180))
    print("Sugar Status:", get_sugar_status(110))
    print("BMI Status:", get_bmi_status(31.5))

if __name__ == "__main__":
    main()
```

Output of the refactored script

```
BP Status: High
BP Status: Normal
Sugar Status: Diabetic
Sugar Status: Normal
BMI Status: Obese
```

The screenshot shows the Visual Studio Code interface with the file explorer on the left displaying a directory named 'LAB13' containing several Python files. The main editor window shows the code for 'task4_after.py'. The code defines three functions: 'get_bp_status', 'get_sugar_status', and 'get_bmi_status', each comparing a reading against a clinical threshold constant. The 'main' function calls these functions with specific values and prints the results. The terminal at the bottom shows the command 'python task4_after.py' and its output: 'BP Status: High', 'BP Status: Normal', 'Sugar Status: Diabetic', 'Sugar Status: Normal', and 'BMI Status: Obese'.

```
1  """Hospital Management System - health status checks using named constants."""
2
3  # ---- Clinical threshold constants (change them here only) ----
4  HIGH_BP_SYSTOLIC = 140      # systolic pressure above this is "High" (mmHg)
5  DIABETIC_SUGAR_LEVEL = 126  # sugar level above this is "Diabetic" (mg/dL)
6  OBESE_BMI_LIMIT = 30        # BMI above this is "Obese" (kg/m^2)
7
8
9  def get_bp_status(systolic):
10     """Return "High" if systolic pressure exceeds HIGH_BP_SYSTOLIC, else "Normal"."""
11     return "High" if systolic > HIGH_BP_SYSTOLIC else "Normal"
12
13
14  def get_sugar_status(sugar_level):
15     """Return "Diabetic" if sugar exceeds DIABETIC_SUGAR_LEVEL, else "Normal"."""
16     return "Diabetic" if sugar_level > DIABETIC_SUGAR_LEVEL else "Normal"
17
18
19  def get_bmi_status(bmi):
20     """Return "Obese" if BMI exceeds OBESE_BMI_LIMIT, else "Normal"."""
21     return "Obese" if bmi > OBESE_BMI_LIMIT else "Normal"
22
23
24  def main():
25     """Print the status for each sample reading."""
26     print("BP Status:", get_bp_status(145))
27     print("BP Status:", get_bp_status(138))
28     print("Sugar Status:", get_sugar_status(180))
29     print("Sugar Status:", get_sugar_status(110))
30     print("BMI Status:", get_bmi_status(31.5))
31
32
33  if __name__ == "__main__":
34     main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

PS D:\AI_Assisted_Coding\Lab13> python task4_after.py
BP Status: High
BP Status: Normal
Sugar Status: Diabetic
Sugar Status: Normal
BMI Status: Obese
PS D:\AI_Assisted_Coding\Lab13> █

x 0 ! 0 Ln 34, Col 1 Spaces: 4 UTF-8 CRLF Python 3.12.3 64-bit

Figure 12: Refactored code (task4_after.py) and its output in VS Code

6. Code Explanation

- **HIGH_BP_SYSTOLIC = 140, DIABETIC_SUGAR_LEVEL = 126, OBESE_BMI_LIMIT = 30** are defined once at the top with unit comments (mmHg, mg/dL, kg/m²).
- `get_bp_status`, `get_sugar_status` and `get_bmi_status` each compare a reading with one constant and return the same label as before ("High", "Diabetic" or "Obese", otherwise "Normal").
- The patient readings stay as arguments, so thresholds (rules) and readings (data) are now clearly separated.
- To change a clinical limit, only the constant at the top needs editing.

7. Verification (Output Unchanged)

Output is identical: BP Status: High / Normal, Sugar Status: Diabetic / Normal, BMI Status: Obese. `verify_all.py` reports PASS for Task 4.

-

Task 5: Refactoring - Enhancing Variable Naming and Code Clarity

2. AI Prompts Used

Prompt 1 - naming and comments

Act as a code reviewer. The script below records a patient's fasting and post-meal sugar test readings, but uses unclear names.

1. Replace non-descriptive variable names with meaningful identifiers.
2. Add inline comments where the logic may be unclear.
3. Keep the output exactly the same.
4. List every assumption you made (such as units) so that I can check it.

[paste legacy code]

4. Original (Legacy) Code and Output

Original code (ta5_before.py)

```
# Legacy code: unclear variable names, no comments
a=90
b=140
c=b-a
print(c)
```

Output of the legacy script

50

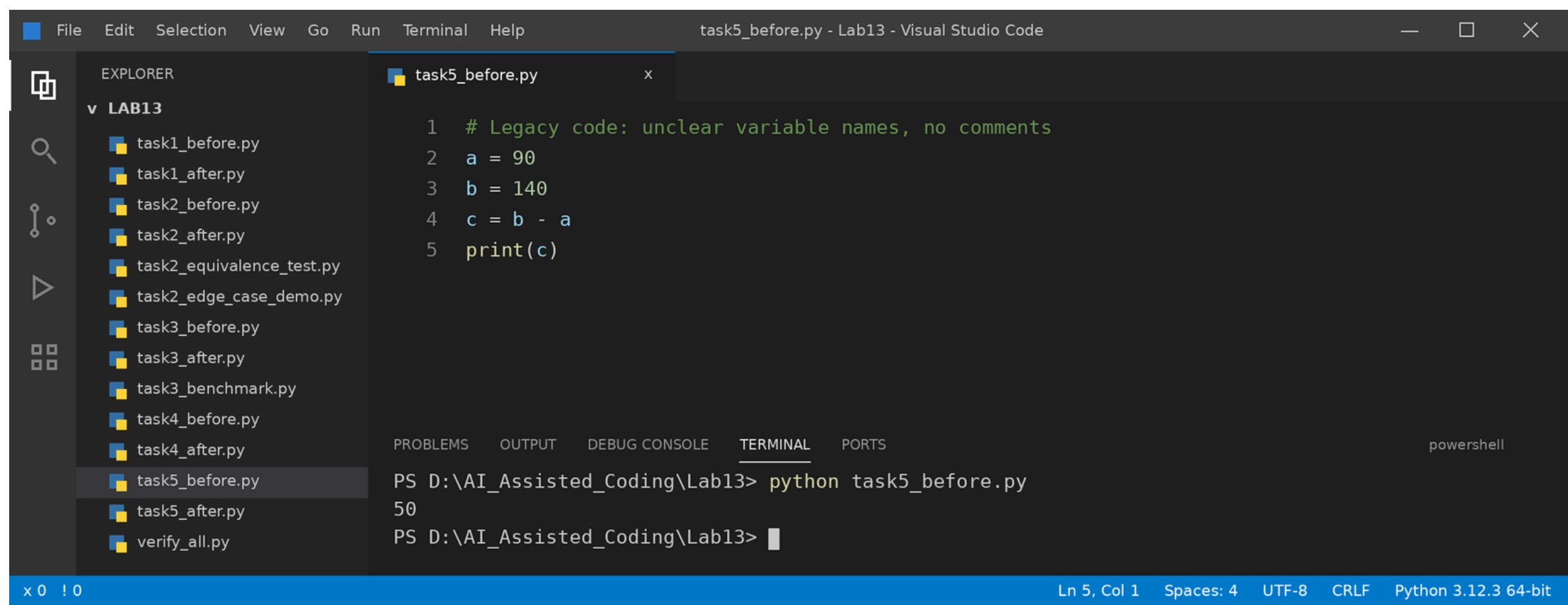


Figure 13: Legacy code (task5_before.py) and its output in VS Code

5. Refactored Code and Output

Refactored code (task5_after.py)

```
# Sugar test readings for one patient (assumed unit: mg/dL)
fasting_sugar = 90    # reading taken after fasting, before any meal
post_meal_sugar = 140 # reading taken after the patient has eaten

# Rise in blood sugar caused by the meal = post-meal reading - fasting reading
sugar_rise = post_meal_sugar - fasting_sugar
```

```
print(sugar_rise)
```

Output of the refactored script

```
50
```

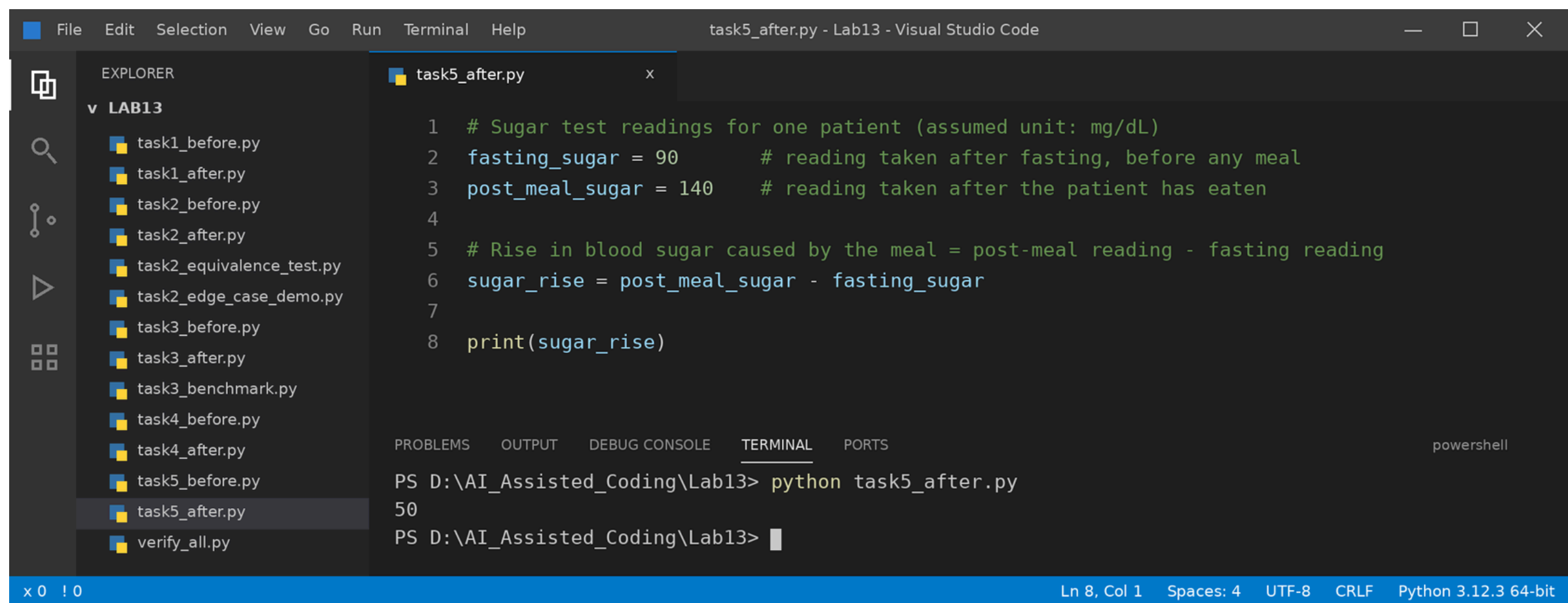


Figure 14: Refactored code (task5_after.py) and its output in VS Code

6. Code Explanation

- a became fasting_sugar and b became post_meal_sugar, which say exactly what each reading is.
- c became sugar_rise (the lab's example name was total_sugar_rised; sugar_rise is the grammatically correct form of the same idea).
- Inline comments explain what each reading is and why the subtraction is done; a header comment records the assumed unit.
- The calculation (post_meal_sugar - fasting_sugar) and the print statement are unchanged.

7. Verification (Output Unchanged)

Both versions print 50. verify_all.py reports PASS for Task 5.

Verification Summary

verify_all.py runs each legacy script and its refactored version in a separate process and compares everything printed to standard output.

verify_all.py

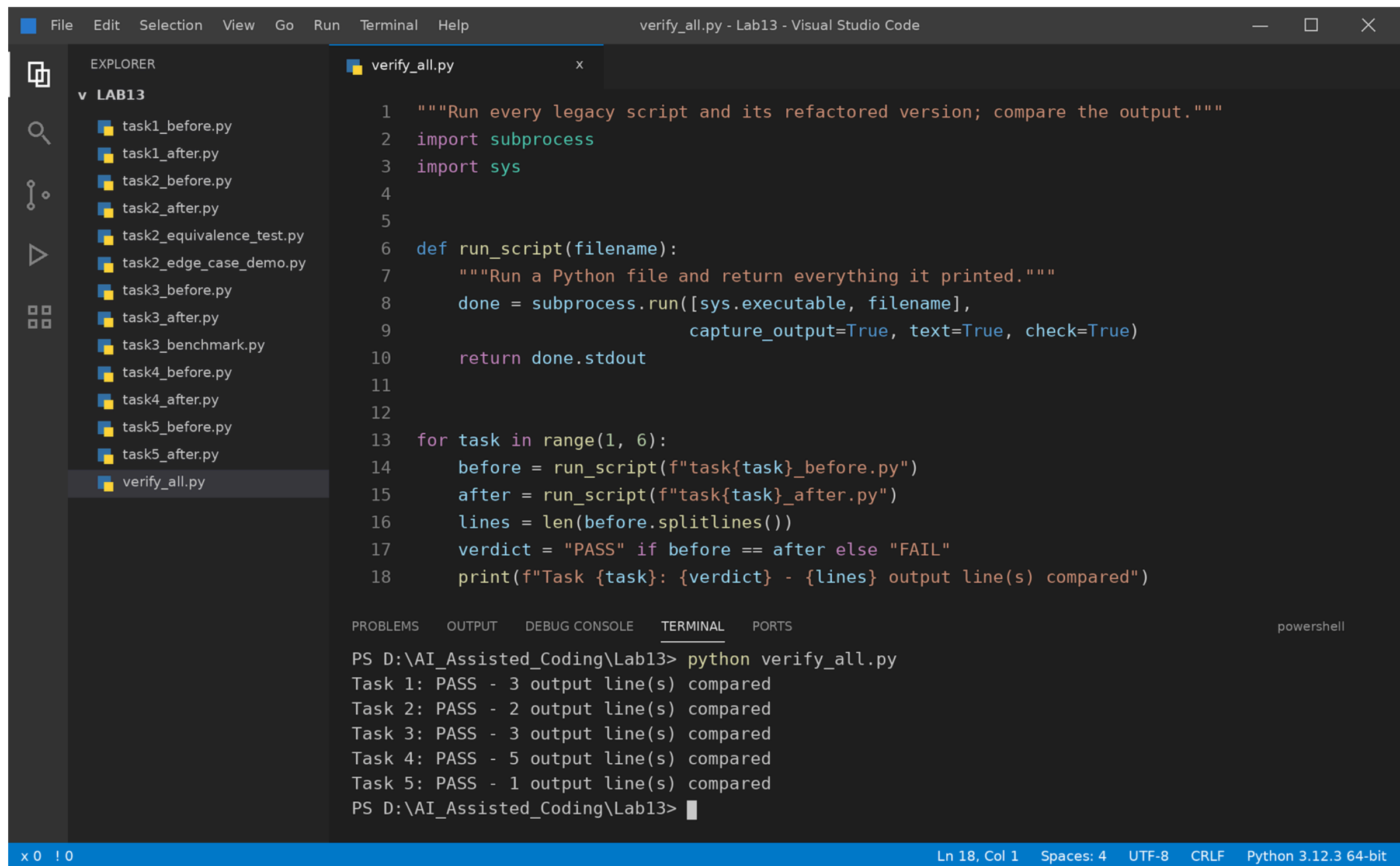
```
"""Run every legacy script and its refactored version; compare the output."""
import subprocess
import sys

def run_script(filename):
    """Run a Python file and return everything it printed."""
    done = subprocess.run([sys.executable, filename],
                          capture_output=True, text=True, check=True)
    return done.stdout

for task in range(1, 6):
    before = run_script(f"task{task}_before.py")
    after = run_script(f"task{task}_after.py")
    lines = len(before.splitlines())
    verdict = "PASS" if before == after else "FAIL"
    print(f"Task {task}: {verdict} - {lines} output line(s) compared")
```

Output

```
Task 1: PASS - 3 output line(s) compared
Task 2: PASS - 2 output line(s) compared
Task 3: PASS - 3 output line(s) compared
Task 4: PASS - 5 output line(s) compared
Task 5: PASS - 1 output line(s) compared
```



The screenshot shows the Visual Studio Code interface. The Explorer pane on the left lists files in a folder named 'LAB13', including task1_before.py, task1_after.py, task2_before.py, task2_after.py, task2_equivalence_test.py, task2_edge_case_demo.py, task3_before.py, task3_after.py, task3_benchmark.py, task4_before.py, task4_after.py, task5_before.py, task5_after.py, and verify_all.py. The main editor displays the contents of verify_all.py, which is a Python script designed to run and compare legacy scripts with their refactored versions. The script includes a docstring, imports subprocess and sys, defines a run_script function, and contains a loop that runs tasks 1 through 5, comparing the output of the legacy and refactored versions. The output of the script is shown in the Terminal pane at the bottom, indicating that all tasks passed the comparison.

```
1 """Run every legacy script and its refactored version; compare the output."""
2 import subprocess
3 import sys
4
5
6 def run_script(filename):
7     """Run a Python file and return everything it printed."""
8     done = subprocess.run([sys.executable, filename],
9                           capture_output=True, text=True, check=True)
10    return done.stdout
11
12
13 for task in range(1, 6):
14     before = run_script(f"task{task}_before.py")
15     after = run_script(f"task{task}_after.py")
16     lines = len(before.splitlines())
17     verdict = "PASS" if before == after else "FAIL"
18     print(f"Task {task}: {verdict} - {lines} output line(s) compared")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS D:\AI_Assisted_Coding\Lab13> python verify_all.py
Task 1: PASS - 3 output line(s) compared
Task 2: PASS - 2 output line(s) compared
Task 3: PASS - 3 output line(s) compared
Task 4: PASS - 5 output line(s) compared
Task 5: PASS - 1 output line(s) compared
PS D:\AI_Assisted_Coding\Lab13>
```

x 0 ! 0 Ln 18, Col 1 Spaces: 4 UTF-8 CRLF Python 3.12.3 64-bit

Findings Worth Noting

- Task 2: the legacy rule uses or in its Stage 1 condition, so some readings (for example 160/70) are labelled Stage 1 although the systolic value alone is in the Stage 2 range. The refactor preserves this on purpose; it is flagged, not fixed.
- Task 3: the set-based lookup shows a large gain only on large data; on the 4-patient sample the difference is not meaningful.
- Task 4: strict > comparisons mean a value exactly equal to a threshold is reported as Normal; the lab asked for unchanged results, so this was kept.

Conclusion

Using AI suggestions together with manual review, the five legacy Hospital Management System snippets were refactored to remove duplication (Tasks 1 and 2), improve lookup performance (Task 3), replace magic numbers with named constants (Task 4) and improve naming and comments (Task 5). Every refactored script produces the same output as its legacy version, each function has a docstring, and the edge cases found during review are documented rather than silently changed.